TiDB

TiDB Transactions on Object Store



Mattias Jonsson
Daniël van Eeden
TiDB, powered by PingCAP

AMS DB meetup, 11 November 2026

THE PROBLEM

An object store is durable and elastic. A commit has to be fast.

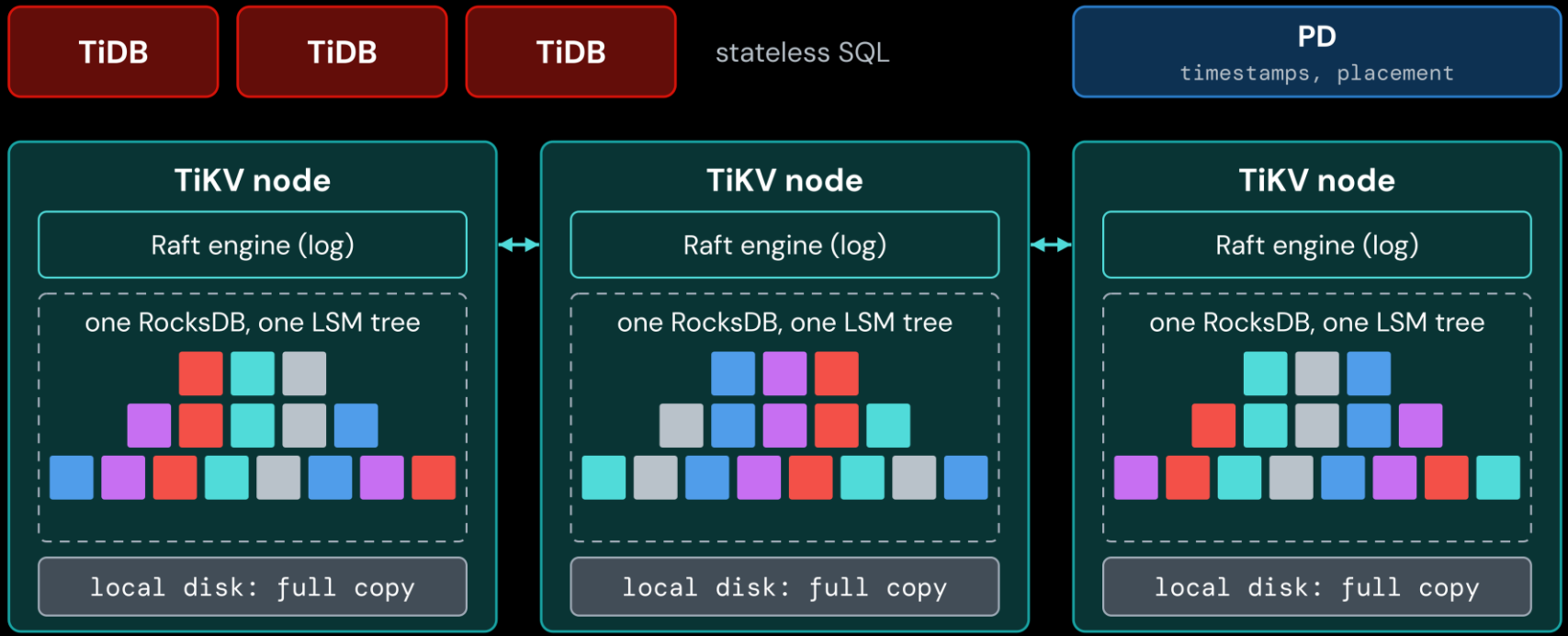
	S3 STANDARD	EBS IO2 BLOCK EXPRESS
Small-request latency	Tens of ms median; 100 to 200 ms as a consistent figure	Under 500 µs average (16 KiB)
Durability	Designed for 11 nines, at least 3 Availability Zones	99.999%, one Availability Zone
Write model	Whole objects, immutable	Blocks, in place, fsync
Capacity	Grows without provisioning	Provisioned per volume
Request rate	3,500 writes, 5,500 reads per second, per prefix	Provisioned IOPS
Charged for	GB stored, plus every request	GB and IOPS provisioned, not per request

A synchronous PUT per commit adds tens of milliseconds and a charge.

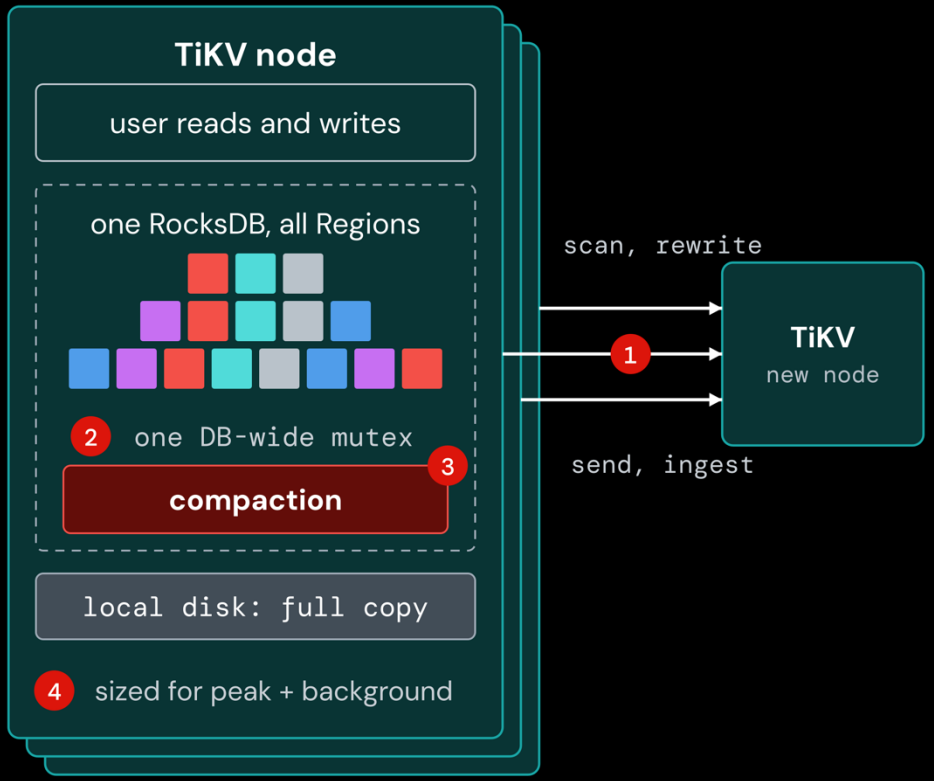
How do you run OLTP transactions on that?

AWS S3 and EBS documentation and pricing pages, read 2026-11-04 and 06. Published figures, not our measurements.

Classic TiDB: every replica has its own full copy



What shared-nothing costs us at scale



1 Scaling moves data

A moved Region is scanned, rewritten as new SST files, sent and ingested, using CPU and I/O on nodes serving traffic.

2 One LSM tree per node

One RocksDB and one DB-wide mutex for all Regions. Contention may start around 100,000 SST files per node.

3 Compaction on serving nodes

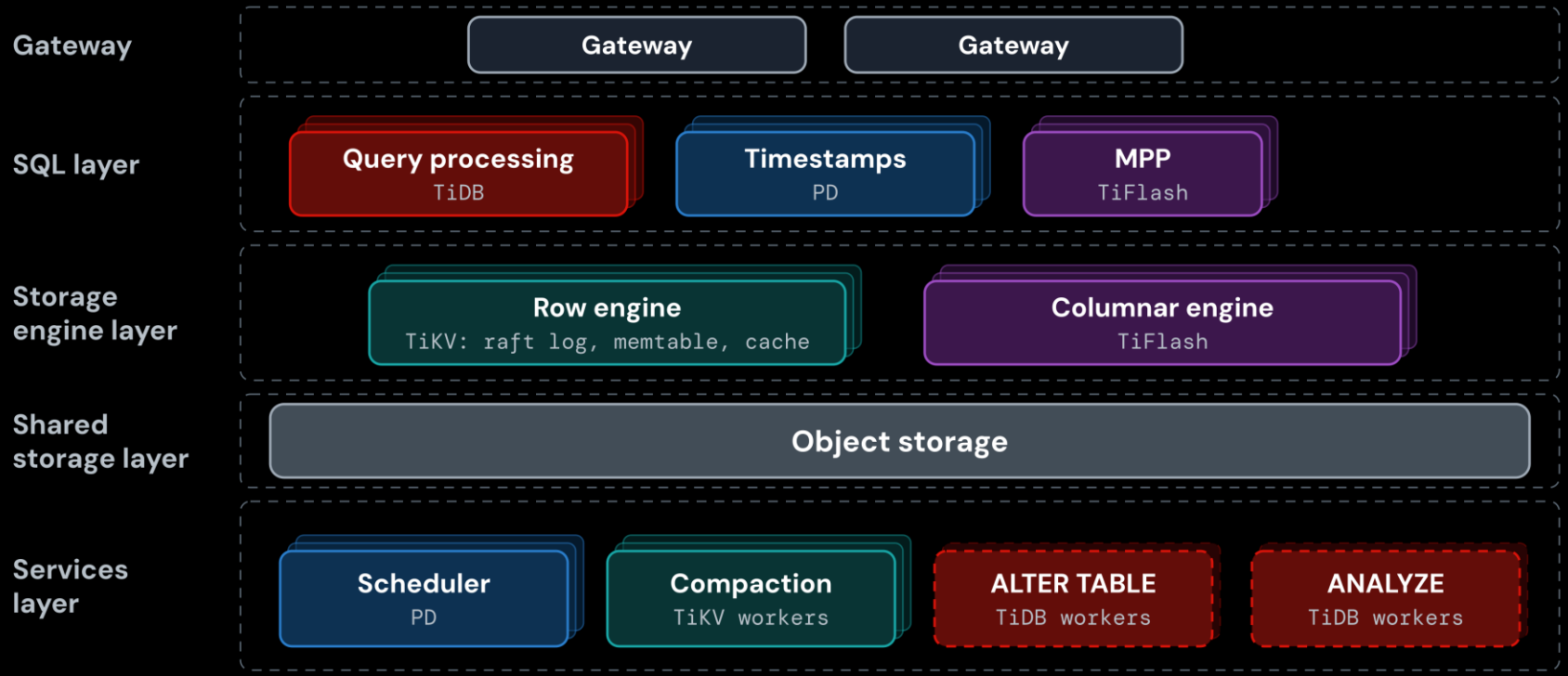
Same CPU and I/O as user queries. If it falls behind, flow control throttles writes.

4 Provision for the peak

Peak traffic plus background work, all the time. Need more storage? Add compute too.

docs.pingcap.com/tidbcloud/tidb-x-architecture; TiKV and RocksDB source; github.com/tikv/tikv/issues/12601

Object storage is the source of truth



Same transactions, different file lifecycle

Kept

MySQL-compatible SQL, optimizer, stateless TiDB servers

Percolator-style transactions and MVCC

Timestamps from PD

Raft replication of Regions

Online DDL state machine

Replaced

RocksDB: KV engine, one LSM tree per Region

Raft Engine: RF engine, log chunks also go to object storage

SST files on local disk: files in object storage, local cache

Background work on serving nodes: elastic worker pools

docs.pingcap.com/tidbcloud/tidb-x-architecture, section "Key innovations"

Three rules follow from that

1. A normal commit writes nothing to the object store.

2. Only immutable files go to the object store. Changes to the LSM tree are Raft entries.

3. One shared durable copy of the data: heavy work runs once, away from the serving nodes.

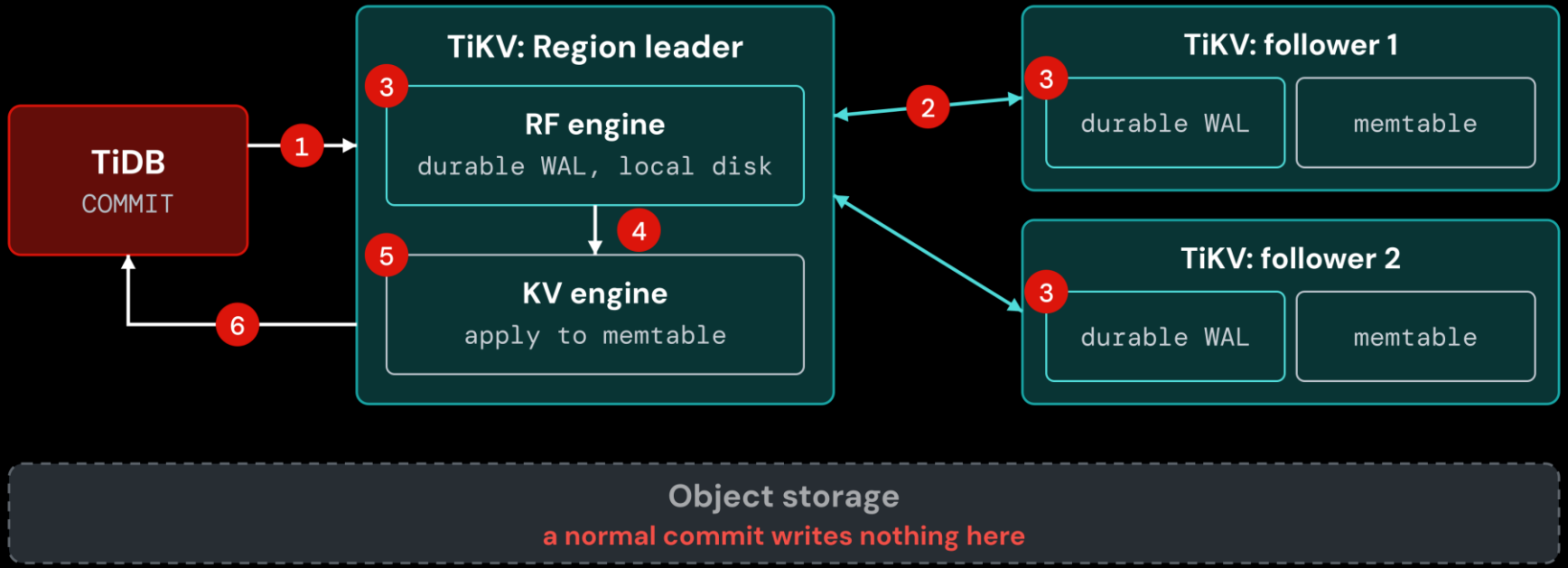
What is written where, and when

WHAT	WRITTEN TO	WHEN	BY
Raft log (write-ahead log)	Local disk	Every write. Commit waits for a majority.	Every replica
Memtable	Memory	Right after the log. Commit waits for the leader.	Every replica
L0 file	Object storage, then local disk	Memtable full. A new memtable takes the writes.	Leader uploads, all download
L1 and lower	Object storage, then local disk	Compaction	Worker uploads, all download
File list (ChangeSet)	Raft log	After each flush or compaction	Leader proposes
Raft log chunks	Object storage	For backup	Each store

Commit waits for it | Background, on serving nodes | Background, on a separate worker

Compaction can fall back to the local node.

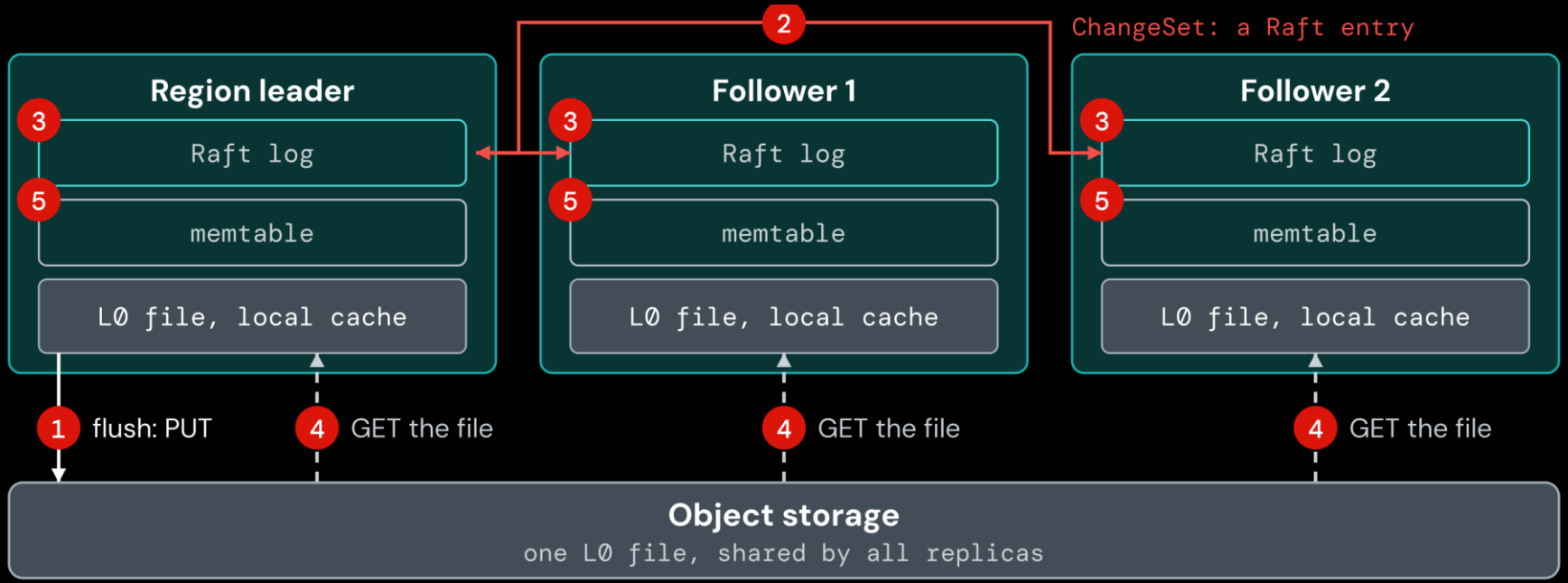
A commit writes nothing to the object store



1 Request reaches the leader
2 Raft entry goes to the followers
3 Each replica persists its WAL
4 Majority persisted: committed
5 Apply to the memtable
6 Acknowledge to TiDB

Exception: apply pauses while a Region loads an ingest or file-based transaction, so a commit can wait on a GET.

Flush: one replica does the work, all get the result



1 Leader builds the L0 file and uploads it

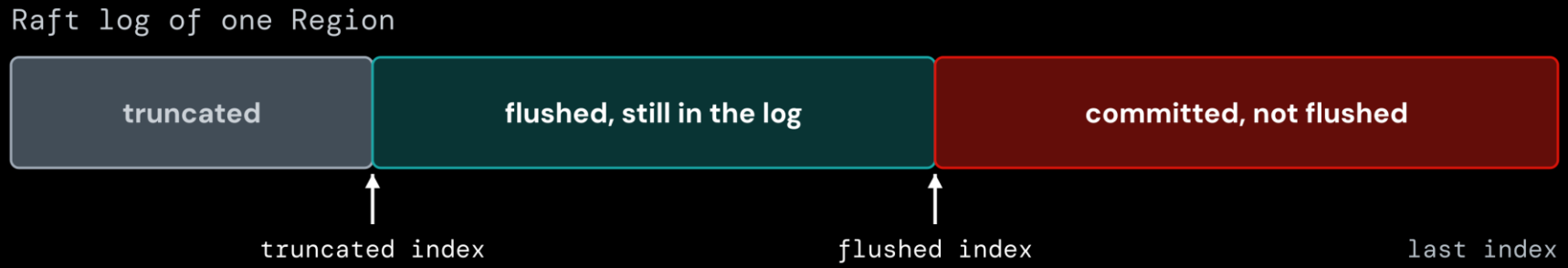
2 Leader proposes a ChangeSet through Raft

3 Majority persists it: committed

4 Every replica downloads the file, then applies

5 Every replica frees the memtable

The Raft log is truncated only after flush

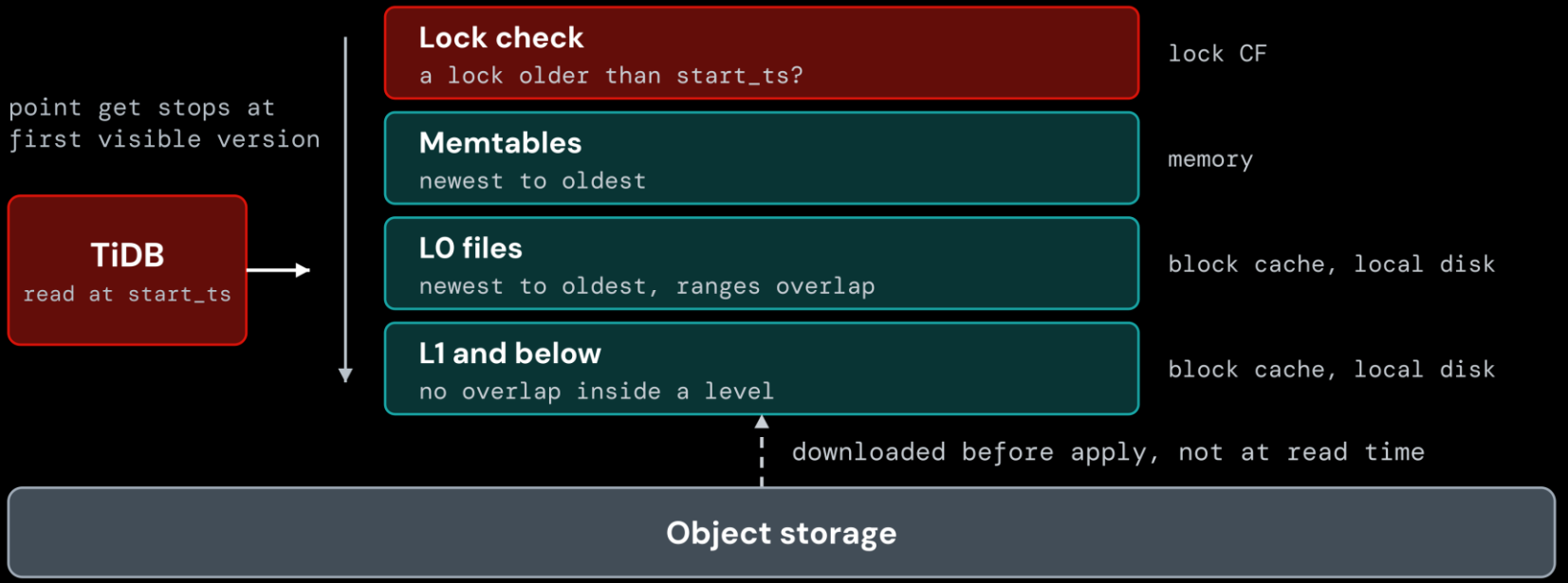



```
truncate_to = min(replicated, applied, flushed)
```

Restart: open the files named in the shard metadata, replay the local log, Raft supplies the rest.

The object store does not make the newest writes safer. The Raft majority does.

Normal reads stay on local disk



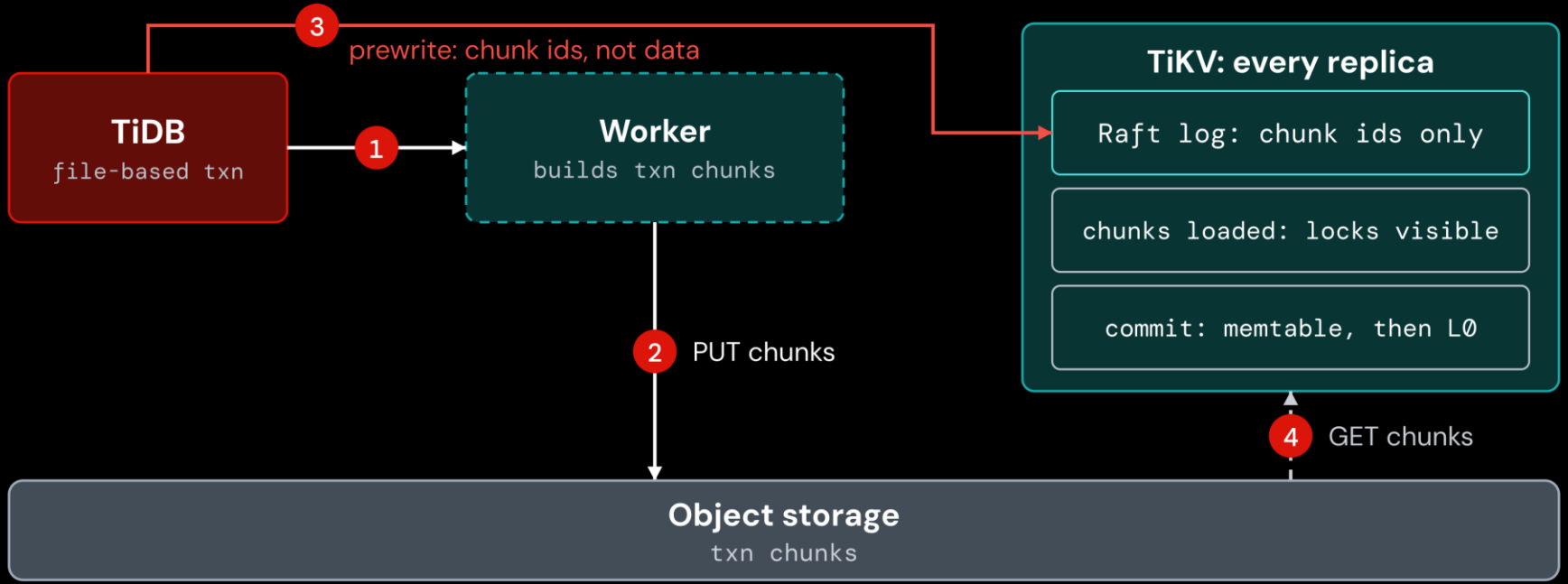
Point get: stops at the first visible version. Range scan: merges all memtables, L0 files and levels.

Leader lease: local read, no Raft round trip. Otherwise read index, same as classic.

Preview: infrequent-access tables keep only metadata local and fetch 1 MiB segments on demand.

docs.pingcap.com/tidbcloud/tiered-storage-overview, tiered-storage-guide

The exception: file-based transactions upload first



Only when enabled, for non-pessimistic transactions with at least 16 MiB of mutations by default.

Gain: lock data stays out of memtables, L0 files and the Raft log.

Cost: that Region's apply stalls while chunks load, and it cannot split or merge until the transaction ends.

From one LSM tree per node to one per Region

Classic: one LSM tree per node

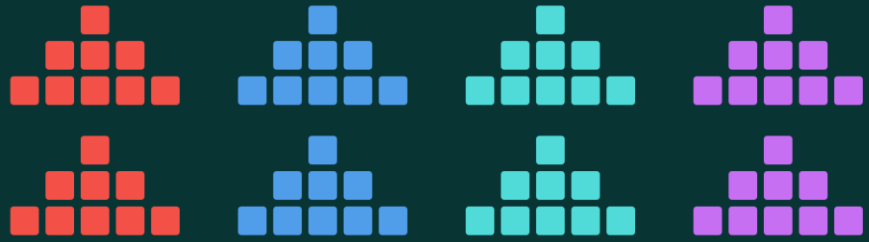


all Regions mixed

one DB-wide mutex

moving a Region: compaction

TiDB X: one LSM tree per Region



each Region compacts alone

no shared tree, no DB-wide mutex

moving a Region: no cross-Region compaction

Larger Regions

Default 96 MiB before v8.4, then 256 MiB. TiDB X: 1 GiB. A move sends metadata, not files.

Old versions apart

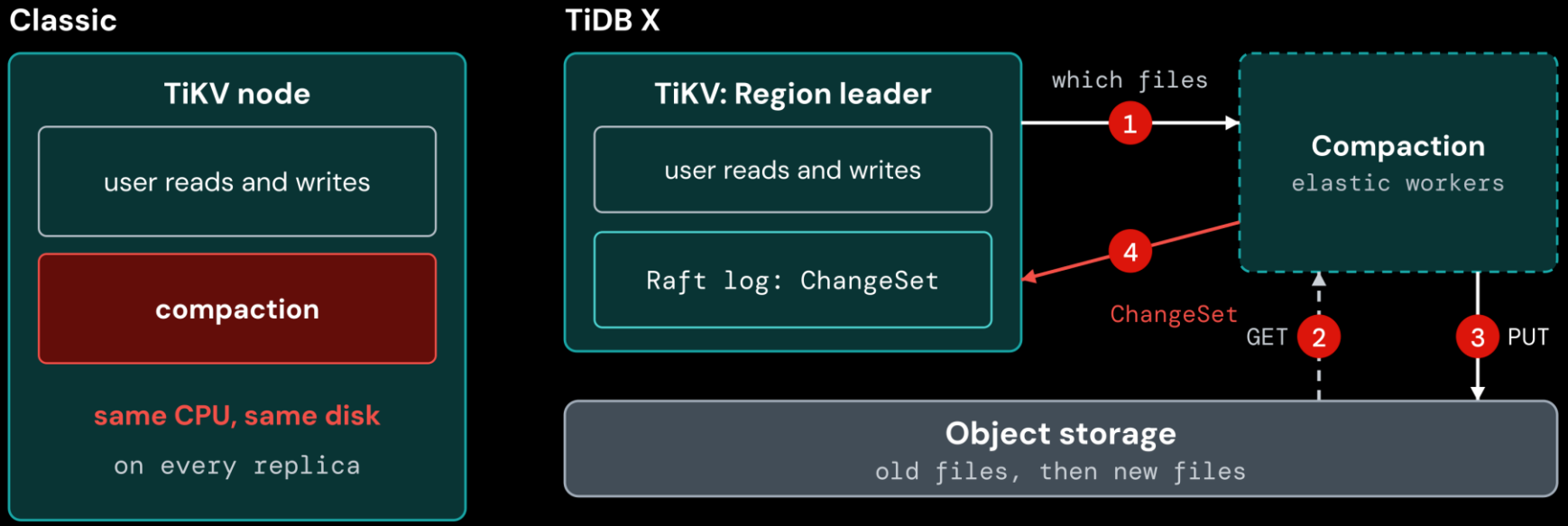
In each file, the newest version of a key is in the data blocks, older versions in separate blocks.

Splits write no files

Children reference the parent's files and read only their own range. Same cost as classic.

Forest and mutex: docs.pingcap.com/tidbcloud/tidb-x-architecture, section "From LSM tree to LSM forest"

Compaction on elastic workers, result through Raft

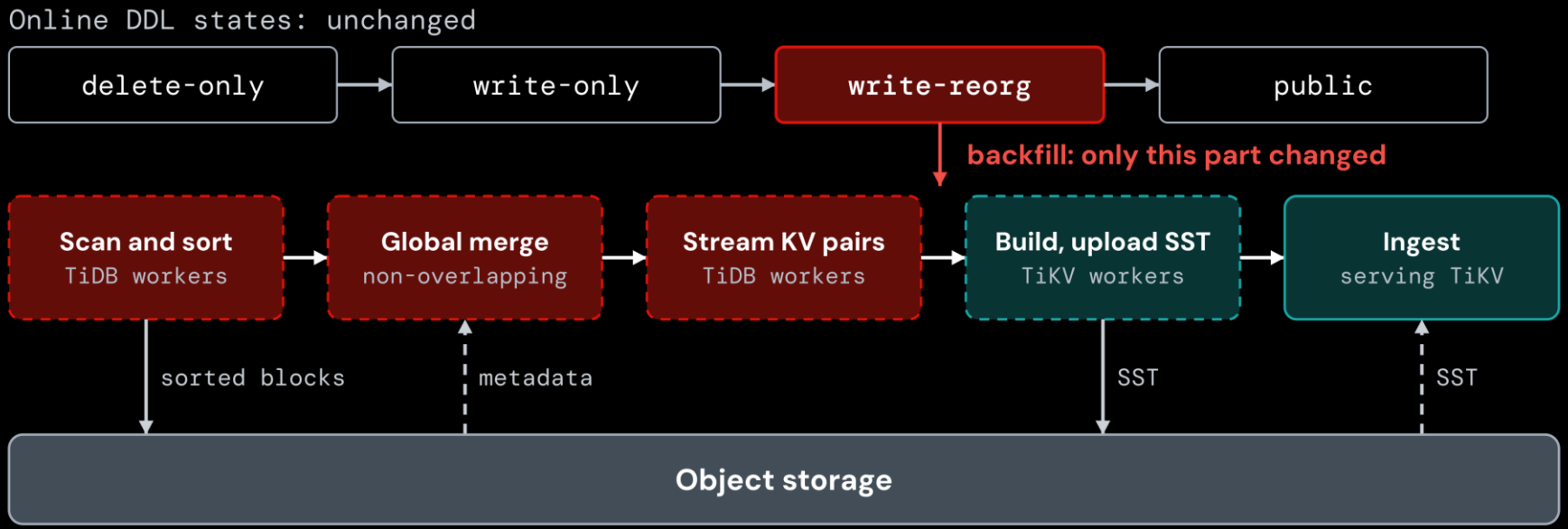


A majority commits the ChangeSet. Every replica then loads the new files and applies it.

Expired MVCC versions are dropped during compaction, using the GC safe point.

Failures retry with backoff, then compaction runs locally if fallback is enabled.

Same online DDL states, different backfill



Workers are provisioned by data size and released afterwards.

Non-overlapping SST files: fewer compactions after ingest.

The download of final files still runs on serving TiKV, in the background.

8 billion rows indexed in 19 minutes

Test 1: one job, 8 billion rows

19 min 10 s

Rows	8 billion
Table size	200 TB
Row size	27 KB
Index	one bigint column
DXF nodes	25, each 8 vCPU, 32 GB

Test 2: 1 billion rows, by method

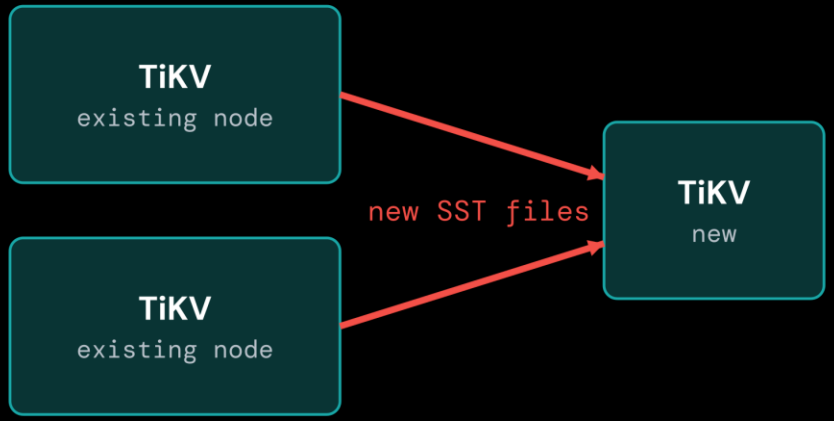
METHOD	NODES	TIME
Transactional DDL	1	12 h 21 min
Fast DDL	1	30 min
Fast DDL + DXF	2	14 to 15 min
Fast DDL + DXF	3	10 min

5x faster than classic, which already made 10x improvement before!

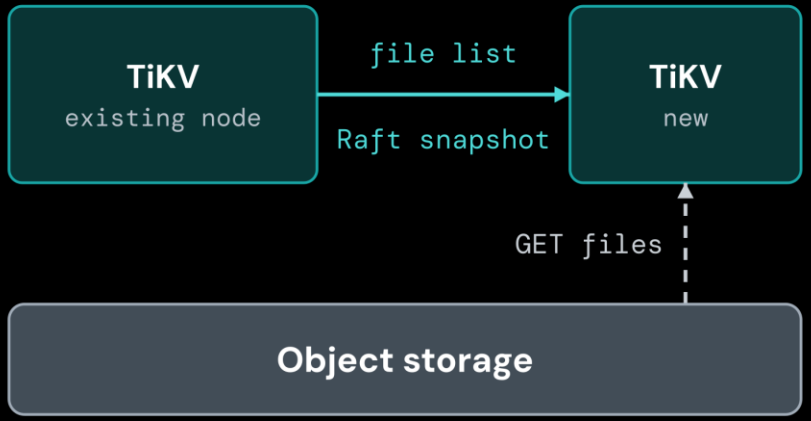
Tests: PingCAP blog, 2026-03-16. 10x: PingCAP blog, 2023-03-09.

Scale-out: peers send metadata, not files

Classic: scan, send, ingest each Region



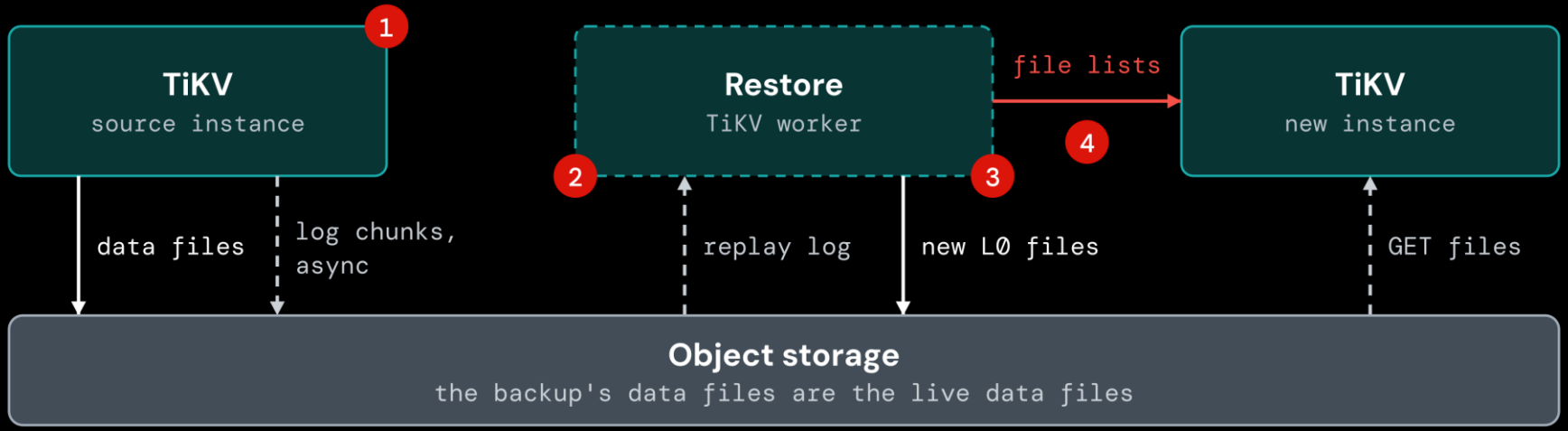
TiDB X: send metadata, load files



Scale-out and scale-in

Peers send no data files. The new replica downloads them; unflushed writes arrive through Raft. 5x to 10x faster.

Backup and restore copy no data files



1 Backup: timestamp, log positions, metadata

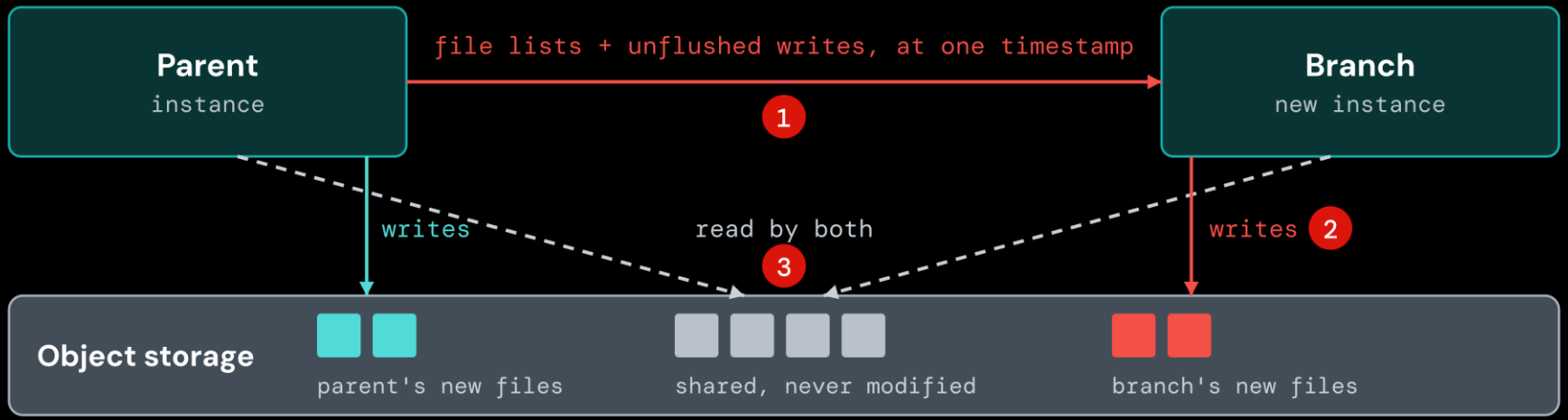
2 Restore rebuilds each Region from its Raft log

3 Flush the log tail, rewrite only newer files

4 New Regions get file lists: files shared, not copied

Backup in seconds, regardless of data size. Kept up to 30 days. Restore: a few minutes.

Branching: a new instance that shares the files



1 Create: file lists plus unflushed writes

2 New writes on either side go to new files

3 A file is deleted when no Region uses it

From now or a past time, up to 14 days.

TRADE-OFFS

What we accepted, and what is still hard

COST	WHY
Fast local disks stay	Raft log, and whole files on every replica. Commit durability depends on them.
Recent writes are not in the object store	Until flush, the durable copy is a Raft majority, same as classic. Log chunks for backup lag behind.
Object store outage	Commits continue at first, then limits throttle writes. If a committed ChangeSet's files cannot be loaded, the store stops.
Harder metadata invariants	ChangeSets must apply in log order on every replica. Out of order, files can be silently dropped.

TAKEAWAYS

What we would tell someone building this

1

Keep object store writes out of the commit path.

Raft log on local disks, truncated only after flush.

2

Replicate metadata, share the files.

ChangeSets through Raft, one file for all replicas.

3

Move heavy work off the serving nodes.

Compaction, index backfill and heavy scans on elastic workers.

THANK YOU

Questions?

READ MORE

TiDB X architecture

docs.pingcap.com/tidbcloud/tidb-x-architecture

The Making of TiDB X

pingcap.com/blog/tidbx-origins-architecture



Daniël on LinkedIn

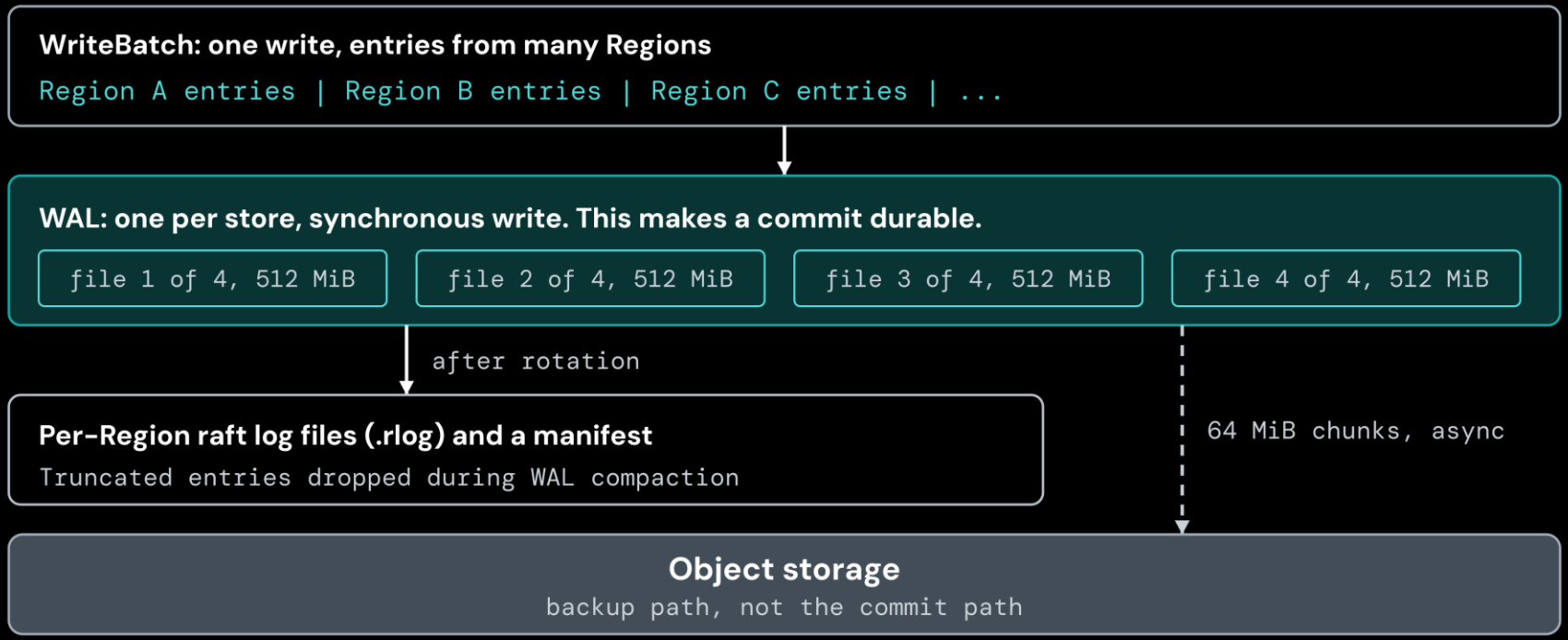


Mattias on LinkedIn



github.com/pingcap/tidb

RF engine: one WAL first, per Region later



Heavy reads: a worker runs the scan



TiKV decides per request, by estimated size, and sends memtable data plus the file list.

The worker is a separate process in a shared pool, not a serving TiDB or TiKV node.

BACKUP SLIDE

What happens when

EVENT	BEHAVIOR
A TiKV node dies	Raft elects new leaders. A replacement replica gets the file list and downloads the files from object storage.
Leader dies during flush	The new leader takes over flushing. Duplicate ChangeSets are skipped.
Raft majority lost	Restore from a snapshot plus uploaded WAL chunks. The last chunk may be missing, fetched from a store if one is reachable.
Object store slow or down	Commits continue at first, then limits throttle writes. If a committed ChangeSet's files cannot be loaded, the store stops.